These problems are about the kernel you built in `34k_processes`, `35k_context_switch` and `36k_scheduling`. There are 3 problems on 5 pages. Answer each part in the space below it, and show your work.

1. **Processes (34k).** Here are `allocproc` and `freeproc` from exercise 34k. `alloc_pid` returns `NEXTPID` and increments it; it starts at 1 after `init()`.

```
pub unsafe fn allocproc() -> *mut Proc {
    for i in 0..NPROC {
        let p = ptr::addr_of_mut!(PROCS[i]);
        if (*p).state == ProcState::Unused {
            (*p).pid = alloc_pid();
            (*p).state = ProcState::Runnable;
            (*p).pagetable = create_pagetable();
            if (*p).pagetable.is_null() {
                return ptr::null_mut();
            }
            return p;
        }
    }
    ptr::null_mut()
}
```

```
pub unsafe fn freeproc(p: *mut Proc) {
    free_pagetable((*p).pagetable);
    (*p).pagetable = ptr::null_mut();
    (*p).pid = 0;
    (*p).state = ProcState::Unused;
}

// NPROC = 4, after init():
let a = allocproc();   // 1
let b = allocproc();   // 2
let c = allocproc();   // 3
freeproc(b);           // 4
let d = allocproc();   // 5
let e = allocproc();   // 6
let f = allocproc();   // 7
```

(a) For `d`, `e` and `f`, give the table slot and pid it refers to (or say it is null). Then give the state and pid of all four slots after line 7.

(b) Now suppose instead the kernel is out of memory at line 6, so `create_pagetable()` returns null. What does `allocproc` return, and what is left in the slot it chose? Why is that a problem for the rest of the kernel? Fix `allocproc`.

2. **Context switch (35k).** `Context` is `#[repr(C)]` with 14 `usize` fields, in order `ra`, `sp`, `s0`, …, `s11`. `swtch(old, new)` stores each of those registers into `*old` (`a0`) with `sd`, loads each from `*new` (`a1`) with `ld`, then `ret`s. The test does:

```
init_context(&mut TASK_CTX, task_entry as usize, stack + PGSIZE); // ra = entry, sp = stack top
swtch(&mut SCHED_CTX, &mut TASK_CTX);    // task_entry sets TASK_RAN, then
                                         //   swtch(&mut TASK_CTX, &mut SCHED_CTX)
check(TASK_RAN == 0xABCD);
```

(a) At what byte offset in a `Context` is `s7` stored? Why does `swtch` save `ra`, `sp` and `s0`–`s11`, but not `t0`–`t6` or `a0`–`a7`?

(b) Kmain's `swtch` runs `ret`. Which code runs next, and on which stack? When does that call to `swtch` finally "return" into `kmain`, and how?

(c) What happens, and why, if:

(i) the restore half loads from `a0` instead of `a1` (`ld ra, 0(a0)`, …)?

(ii) `init_context` is passed `stack` instead of `stack + PGSIZE`?

(iii) `#[repr(C)]` is removed from `Context`?

3. **Scheduling (36k).** This is the round-robin policy from 36k. After each turn the process yields with `swtch` and the scheduler loop marks it `Runnable` again.

```
fn pick_next(&mut self, states: &[ProcState]) -> Option<usize> {
    let n = states.len();
    (0..n).map(|off| (self.next + off) % n)
          .find(|&i| states[i] == ProcState::Runnable)
          .map(|i| { self.next = (i + 1) % n; i })
}
```

(a) With `self.next = 0`, a 5-slot table starts as

`[A: Runnable, B: Sleeping, C: Runnable, -: Unused, D: Runnable]`

Right after the third turn, `B` wakes up and becomes `Runnable`. Fill in the table for the first 7 turns.

Turn	Index `pick_next` returns	Process that runs	`self.next` after
1			
2			
3			
B wakes up			
4			
5			
6			
7			

(b) The 36k test runs pids 1, 3, 4 (pid 2 sleeps) for 3 turns each and expects `1 3 4 1 3 4 1 3 4`. What order do you get if `pick_next` sets `self.next = i` instead of `(i + 1) % n`? Why?

(c) Suppose pid 3's code had an infinite loop before its first yield. What happens to pids 1 and 4, and why? What would fix it?